



UNIVERSITY OF PISA
MASTER'S DEGREE IN CYBERSECURITY

LLM–NMAP: NETWORK SCANNING USING LARGE LANGUAGE MODELS

COURSE: NETWORK SECURITY – UNIVERSITY OF PISA
PROFESSORS: ROSARIO GARROPPA, MICHELE PAGANO

Academic year 2025/2026

Author(s)
Aurélien Roumégoux
Hatem Elashmawy

Abstract

This report describes the design and implementation of a small framework that integrates Nmap with a Large Language Model (LLM) to perform network scans based on natural-language requests. The framework is built around the `llm` command-line tool, the `llm-tools-nmap.py` plugin, and Google Gemini as the LLM provider, running on a Kali Linux virtual machine inside a VirtualBox lab.

We document how to set up the entire environment, from the virtual network to the LLM configuration, and we present several scenarios where the user describes the desired network analysis to the LLM. The LLM then selects and runs appropriate Nmap commands, the framework collects the scan results, and the output is presented back to the user in a human-readable form. Comparisons with manual Nmap scans are used to validate the correctness of the LLM-driven scans.

1 Introduction

Nmap is a widely used network scanner that supports a rich set of options, scan types, and output formats. While powerful, it can be difficult for less experienced users to remember the right combination of flags and parameters for a given analysis task.

The goal of this project is to explore how a Large Language Model (LLM) can act as an “interpreter” between a human operator and Nmap. Instead of manually constructing the command line, the user describes the analysis in natural language (e.g., “scan this host and show me which services are running”), and the LLM:

1. Interprets the request,
2. Chooses an appropriate Nmap function and parameters (using `llm-tools-nmap.py`),
3. Executes Nmap,
4. Returns the results and optionally a short summary.

The project specification and the professor’s email define two main requirements:

- Produce a **technical report** that describes how to **set up the entire framework**.
- Provide a **demo** that shows that, when the analysis is described to the LLM, the system:
 1. Correctly configures and runs Nmap,
 2. Acquires the data obtained from Nmap,
 3. Presents the results to the user.

This report addresses these requirements by documenting our architecture, setup steps, scenarios, and a brief evaluation.

2 Background

2.1 Nmap

Nmap is a command-line tool used for network discovery and security auditing. It can perform host discovery, port scanning, service and version detection, and more. Typical usage in our environment involves commands such as:

- `nmap 10.0.2.15` – basic scan of a single host in our NAT network
- `nmap -sV 10.0.2.15` – service and version detection on the same host
- `nmap -sn 10.0.2.0/24` – host discovery on the NAT subnet

In practice, a user must know which flags to use for their goal; this is precisely what we try to offload to the LLM.

2.2 LLMs and the `llm CLI`

Large Language Models (LLMs) such as Gemini and GPT can understand and generate text, and can also be integrated with tools via “function calling” or plugins. The `llm CLI` provides:

- A unified command-line interface to multiple LLM providers.
- A plugin system to extend LLMs with custom tools (e.g. Nmap).

In our project, the `llm CLI` acts as the entry point for the user. The user types a natural-language prompt, and `llm` forwards it—together with the function definitions from `llm-tools-nmap.py`—to Gemini.

2.3 `llm-tools-nmap.py`

The `llm-tools-nmap.py` plugin defines one or more “functions” that the LLM can call when it decides that a network scan is needed. Each function corresponds to a certain Nmap operation (e.g. a “quick scan” or a “ping scan”).

When the user prompt suggests a scan (for example, “Scan my local network to find live hosts with ping”), the LLM chooses the appropriate function, provides arguments (target, scan type), and the plugin executes Nmap and returns the output. That output is then included in the final LLM response shown to the user.

3 System Architecture and Environment

3.1 Overview

Our architecture is based on a small VirtualBox lab plus one physical host:

- **Kali Linux VM** (controller):
 - Runs the `llm CLI`.
 - Uses Google Gemini via `llm-gemini`.
 - Loads `llm-tools-nmap.py` to connect the LLM to Nmap.
 - Executes all Nmap scans.
- **Ubuntu Server VM**:
 - “Classic” target with specific services (e.g. SSH) that we can enable/disable.
 - Used to demonstrate how open ports appear or disappear in the scans.
- **BeeBox VM**:
 - Vulnerable target with many open ports and services by default.
 - Used as a richer scenario to show how LLM–Nmap behaves on a more complex target.

- **Mac laptop (physical):**
 - Physical host on the same network.
 - Used to show that the framework is not limited to virtual machines, as long as the target is reachable and we have permission to scan it.

3.2 VirtualBox Network Setup

We use a **VirtualBox NAT Network** so that the VMs can communicate with each other on a private subnet.

3.2.1 Creating a NAT Network

1. Open VirtualBox.
2. Go to **Tools** → **NAT Networks**.
3. Click **Create** and configure:
 - A name (e.g. NSProject),
 - An IP range (e.g. 10.0.2.0/24).

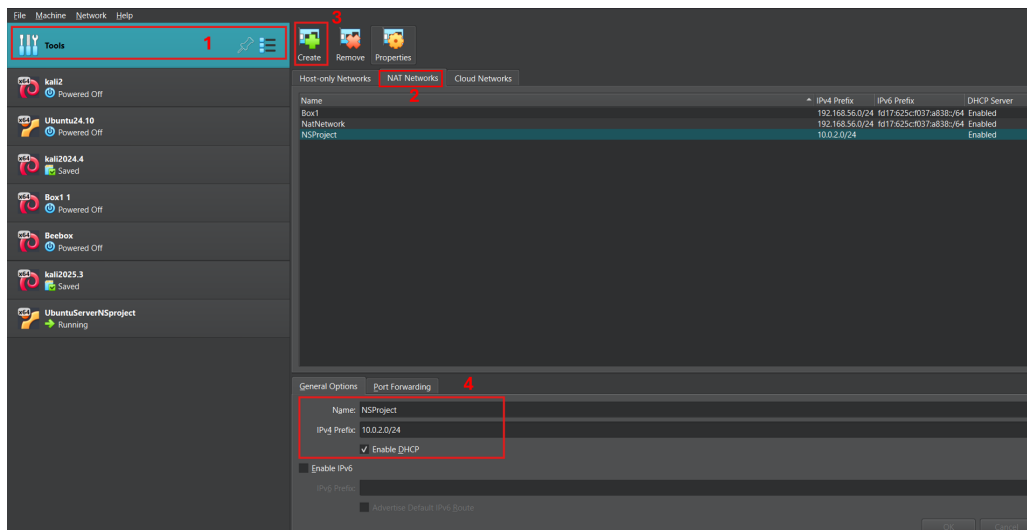


Figure 1: Create NAT Network

3.2.2 Attaching VMs to the NAT Network

For each VM (Kali, Ubuntu Server, BeeBox):

1. Open **Settings** → **Network**.
2. Set **Attached to** = NAT Network.
3. Select the NAT network created earlier (NSProject).

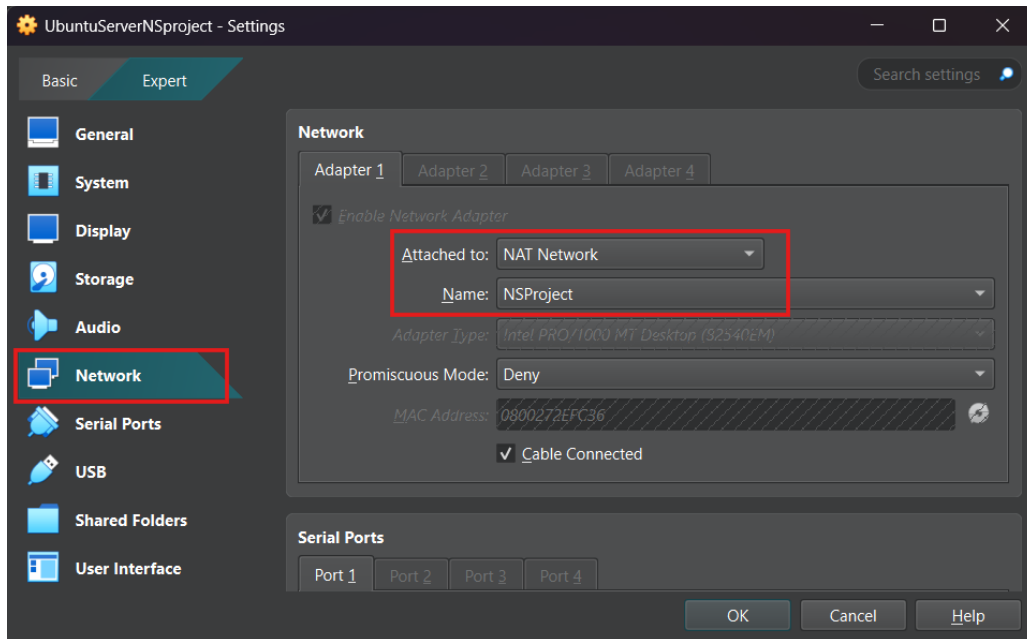


Figure 2: Add VMs to NAT Network

3.2.3 Verifying Connectivity

On each VM, retrieve its IP address:

```
ifconfig
```

From the Kali VM, test reachability:

```
ping 10.0.2.15 # Ubuntu Server in our example
```

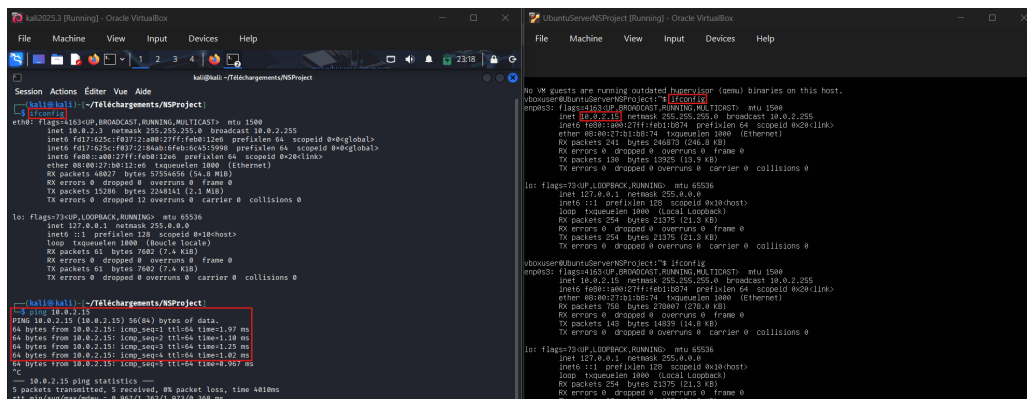


Figure 3: Kali Ping to Ubuntu Serveur

```
ping 10.0.2.7 # Beebox in our example
```

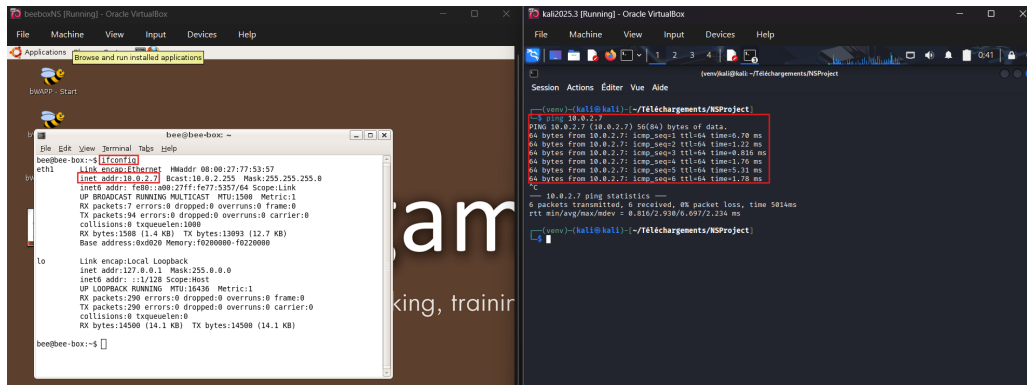


Figure 4: Kali Ping to Beebox

If the pings succeed, the VMs are on the same network and reachable.

3.3 Roles of Each Machine

- **Kali VM:** Runs LLM-Nmap (LLM + `llm-tools-nmap.py` + Nmap). Used to issue all LLM-driven scans and manual Nmap scans.
- **Ubuntu Server VM:** Initially has no open services; then we install and enable `openssh-server`. Allows us to show the difference between “no open ports” and “port 22 open” on **10.0.2.15**.
- **BeeBox VM:** Has many open ports and vulnerable services by design. Used to generate more complex Nmap outputs (Scenario 3).
- **Mac laptop:** Serves as an external target reachable from the Kali VM. Demonstrates that the framework works across a real network.

4 LLM-Nmap Setup on Kali

NOTE: For the following part, we will only use the Ubuntu Server as a target, but it also works for the Beebox VM or any VM/real computer you use.

This section describes the concrete steps to configure the LLM-Nmap framework on the Kali VM.

4.1 Installing llm

On Kali, create a virtual environment and install `llm`:

```
python3 -m venv <name of your venv> # create a virtual environment
source <name of your venv>/bin/activate # activate the virtual environment
pip install llm # install llm
```

```

(kali@kali)-[~/Téléchargements/NSProject]
$ python3 -m venv venv

(kali@kali)-[~/Téléchargements/NSProject]
$ source venv/bin/activate

(venv)-(kali@kali)-[~/Téléchargements/NSProject]
$ pip install llm
Collecting llm
  Downloading llm-0.27.1-py3-none-any.whl.metadata (30 kB)
Collecting click (from llm)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting condense-json≥0.1.3 (from llm)
  Downloading condense_json-0.1.3-py3-none-any.whl.metadata (4.4 kB)
Collecting openai≥1.55.3 (from llm)
  Downloading openai-2.8.1-py3-none-any.whl.metadata (29 kB)
Collecting click-default-group≥1.2.3 (from llm)
  Downloading click_default_group-1.2.4-py2.py3-none-any.whl.metadata (2.8 kB)

```

Figure 5: Set Up LLM

4.2 Configuring Gemini

With *llm* we can use a lot of different llms such as ChatGPT or Gemini. We decided to use Google Gemini as the LLM provider because the API is available for free.

1. Install the Gemini plugin:

```
llm install llm-gemini
```

2. Set the Gemini API key (obtained from [Google AI Studio](#)):

```
llm keys set gemini
```

3. Test Gemini:

```
llm -m gemini-2.0-flash "Hello"
```

```

(venv)-(kali@kali)-[~/Téléchargements/NSProject]
$ llm install llm-gemini
Collecting llm-gemini
  Downloading llm-gemini-0.27-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: llm≥0.27 in ./venv/lib/python3.13/site-packages (from llm-gemini) (0.27.1)
Requirement already satisfied: httpx in ./venv/lib/python3.13/site-packages (from llm-gemini) (0.28.1)
Collecting ijson (from llm-gemini)
  Downloading ijson-3.4.0.post0-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (23 kB)
Requirement already satisfied: click in ./venv/lib/python3.13/site-packages (from llm≥0.27→llm-gemini) (8.3.1)
Requirement already satisfied: condense-json≥0.1.3 in ./venv/lib/python3.13/site-packages (from llm≥0.27→llm-gemini) (0.1.3)

```

Figure 6: Setup Gemini – Test 1

```

(venv)-(kali@kali)-[~/Téléchargements/NSProject]
$ llm keys set gemini
Enter key:

(venv)-(kali@kali)-[~/Téléchargements/NSProject]
$ llm -m gemini-2.0-flash 'Hello'
Hello! How can I help you today?

```

Figure 7: Setup Gemini – Test 2

If a valid text response is returned, the LLM configuration is working.

4.3 Adding llm-tools-nmap.py

To perform a request to Nmap through an LLM, we need to add the Python tool *llm-tools-nmap.py*:

1. Download *llm-tools-nmap.py* from the reference repository: <https://github.com/peter-hackertarget/llm-tools-nmap>
2. Place the file (e.g.) in `/home/kali/LLM-Nmap/` or in your project directory on Kali.
3. From that directory, run a first test:

```
llm -m gemini-2.0-flash --functions llm-tools-nmap.py \  
"Scan my local network to find live hosts with ping"
```

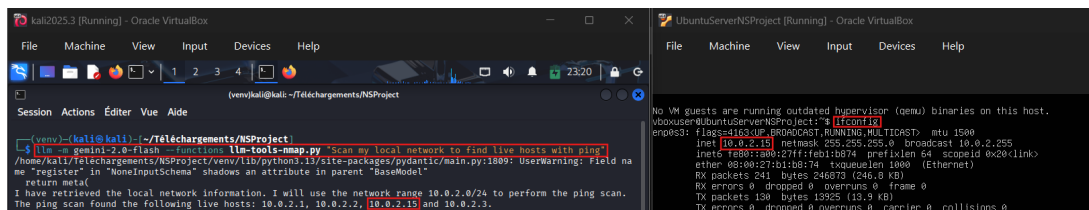


Figure 8: LLM-Nmap – Scan network with ping

In our setup, the **Ubuntu Server** VM is detected by the tool.

NOTE: The Beebox isn't detected because the VM was shut down when the screenshot was taken.

Then, we perform a more specific scan of the Ubuntu Server at IP **10.0.2.15**:

```
llm -m gemini-2.0-flash --functions llm-tools-nmap.py \  
"Make a quick scan of 10.0.2.15"
```

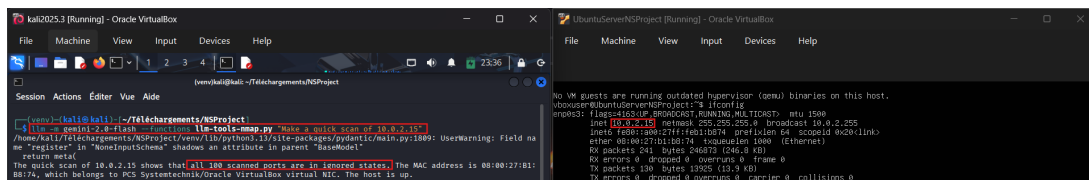


Figure 9: LLM-Nmap – Quick scan, no open ports

At this stage, no open ports are found. This is expected, because we have not yet enabled any service on the Ubuntu Server.

5 Scenarios and Implementation Details

We implemented and tested three main scenarios to demonstrate the behaviour of LLM-Nmap.

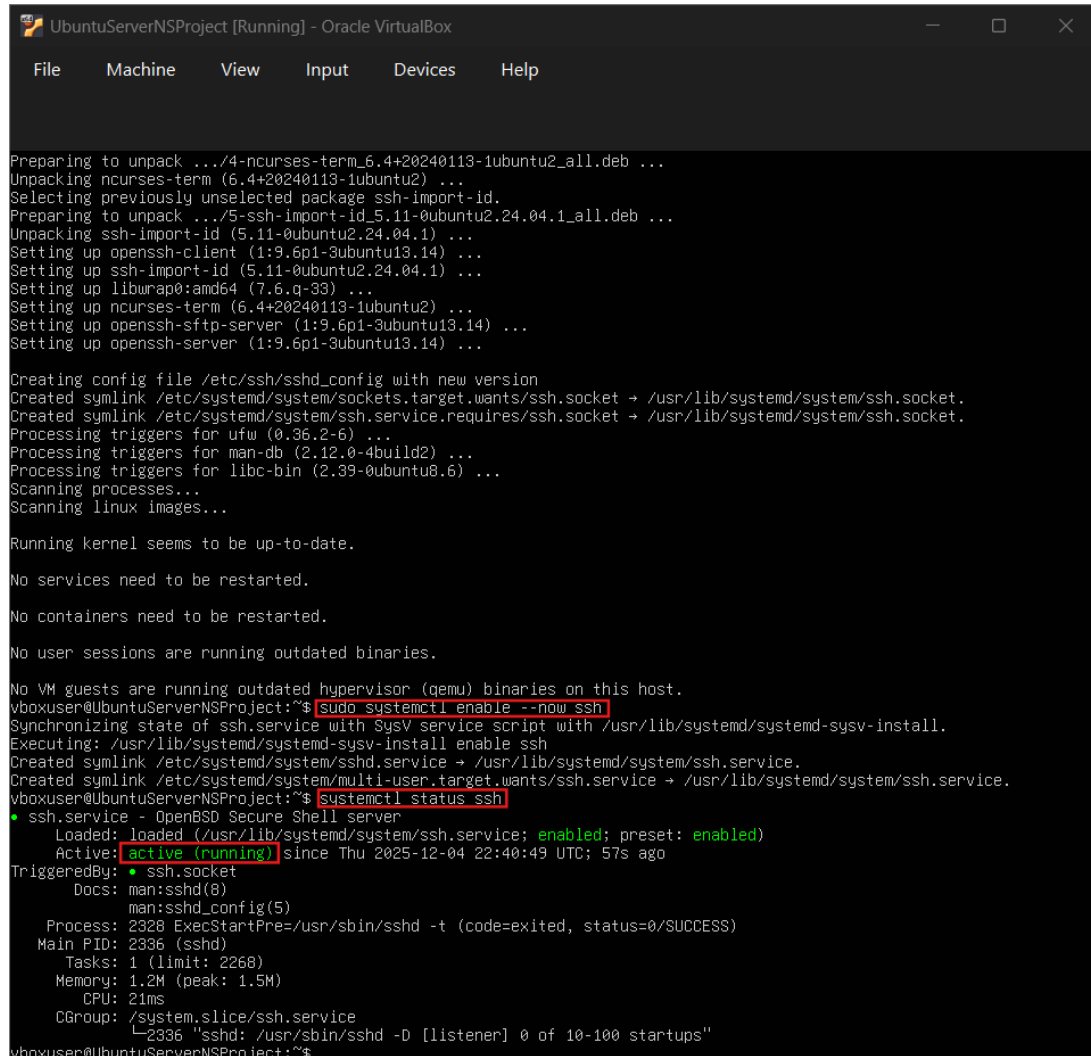
5.1 Scenario 1 – Kali VM Scanning Ubuntu Server VM (10.0.2.15)

5.1.1 Opening SSH on Ubuntu Server

We previously saw that no ports were open. Let's open a port (e.g. SSH) to see if it will be detected by *llm-nmap*.

On the Ubuntu Server VM (10.0.2.15):

```
sudo apt update
sudo apt install openssh-server -y
sudo systemctl enable --now ssh
systemctl status ssh
```



```
UbuntuServerNSProject [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Preparing to unpack .../4-ncurses-term_6.4+20240113-1ubuntu2_all.deb ...
Unpacking ncurses-term (6.4+20240113-1ubuntu2) ...
Selecting previously unselected package ssh-import-id.
Preparing to unpack .../5-ssh-import-id_5.11-0ubuntu2.24.04.1_all.deb ...
Unpacking ssh-import-id (5.11-0ubuntu2.24.04.1) ...
Setting up openssh-client (1:9.6p1-3ubuntu13.14) ...
Setting up ssh-import-id (5.11-0ubuntu2.24.04.1) ...
Setting up liburip0:amd64 (7.6.0-33) ...
Setting up ncurses-term (6.4+20240113-1ubuntu2) ...
Setting up openssh-sftp-server (1:9.6p1-3ubuntu13.14) ...
Setting up openssh-server (1:9.6p1-3ubuntu13.14) ...

Creating config file /etc/ssh/sshd_config with new version
Created symlink /etc/systemd/system/sockets.target.wants/ssh.socket → /usr/lib/systemd/system/ssh.socket.
Created symlink /etc/systemd/system/ssh.service.requires/ssh.socket → /usr/lib/systemd/system/ssh.socket.
Processing triggers for ufw (0.36.2-6) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.6) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
vboxuser@UbuntuServerNSProject:~$ sudo systemctl enable --now ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/ssh.service → /usr/lib/systemd/system/ssh.service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /usr/lib/systemd/system/ssh.service.
vboxuser@UbuntuServerNSProject:~$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-12-04 22:40:49 UTC; 57s ago
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 2328 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 2336 (sshd)
     Tasks: 1 (limit: 2268)
    Memory: 1.2M (peak: 1.5M)
       CPU: 21ms
   CGroup: /system.slice/ssh.service
           └─2336 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

vboxuser@UbuntuServerNSProject:~$
```

Figure 10: Ubuntu Server – Open SSH port

After this, port **22/tcp** should be open on 10.0.2.15.

5.1.2 LLM–Nmap Scan After Enabling SSH

We re-run the quick scan from Kali:

```
llm -m gemini-2.0-flash --functions llm-tools-nmap.py \
    "Make a quick scan of 10.0.2.15"
```

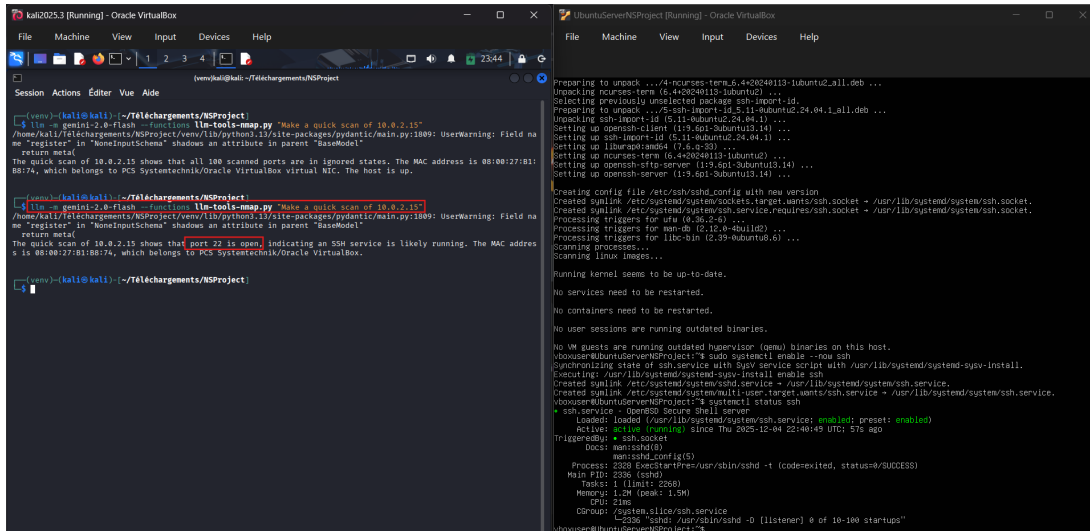


Figure 11: LLM–Nmap – Quick scan, SSH port open

Port 22/tcp is now detected as open.

5.1.3 Comparison with Manual Nmap

To validate the result, we run a manual Nmap scan from Kali:

```
nmap 10.0.2.15
```

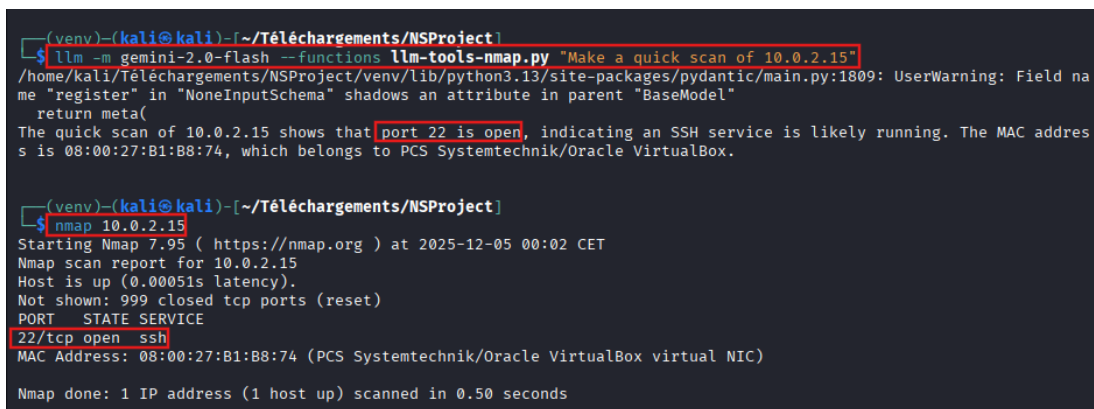


Figure 12: LLM–Nmap vs Nmap – Quick scan, SSH port open

The manual output confirms that port 22 is open and identifies the SSH service. This verifies that:

- The LLM correctly configured and executed Nmap via the plugin, and
- The data presented by LLM–Nmap is consistent with the “ground truth”.

5.2 Scenario 2 – Kali VM Scanning a Physical Mac Host (10.184.43.104)

In this scenario, Kali runs as a VM on one laptop, and the target is a Mac laptop on the same Wi-Fi network.

5.2.1 Target: Mac Host

On the Mac, we identify:

- The IP address (shown in the screenshot).
- The open ports (e.g. enabled services).

```
nd6 options=201<PERFORMNUD,DAD>
tommy@Tommys-MacBook-Air ~ % netstat -an | grep LISTEN
tcp6      0      0  *.64322          *.*          LISTEN
tcp4      0      0  *.64322          *.*          LISTEN
tcp6      0      0  *.5000           *.*          LISTEN
tcp4      0      0  *.5000           *.*          LISTEN
tcp6      0      0  *.7000           *.*          LISTEN
tcp4      0      0  *.7000           *.*          LISTEN
tcp46     0      0  *.53378          *.*          LISTEN
tommy@Tommys-MacBook-Air ~ %
```

Figure 13: Mac Open Ports

Figure: Screenshot showing the open ports on the Mac laptop

```
status: inactive
en0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
options=6460<TS04,TS06,CHANNEL_IO,PARTIAL_CSUM,ZEROINVERT_CSUM>
ether
inet6                prefixlen 64 secured scopeid 0xb
inet 10.184.43.104 netmask 0xffffff00 broadcast 10.184.43.255
nd6 options=201<PERFORMNUD,DAD>
media: autoselect
status: active
```

Figure 14: Mac IP Address

Figure: Screenshot showing the IP address of the Mac laptop

5.2.2 Nmap and LLM–Nmap Scans from Kali

From the Kali VM, we run:

Manual scan:

```
nmap -A 10.184.43.104
```

```

(kali㉿kali)-[~/Téléchargements/NSProject]
$ nmap -A 10.184.43.104
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-05 15:49 CET
Nmap scan report for 10.184.43.104
Host is up (0.0018s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
5000/tcp  open  rtsp
|_rtsp-methods: ERROR: Script execution failed (use -d to debug)
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.1 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/920.10.1
|     X-Apple-ProcessingTime: 1
|     X-Apple-RequestReceivedTimestamp: 96100899
|   GetRequest:
|     HTTP/1.1 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/920.10.1
|     X-Apple-ProcessingTime: 2
|     X-Apple-RequestReceivedTimestamp: 96095768
|   HTTPOptions:
|     HTTP/1.1 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/920.10.1
|     X-Apple-ProcessingTime: 3
|     X-Apple-RequestReceivedTimestamp: 96100866
|   RTSPRequest:
|     RTSP/1.0 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/920.10.1
|     X-Apple-ProcessingTime: 3
|     X-Apple-RequestReceivedTimestamp: 96095852
|   SIPOptions:
|     RTSP/1.0 403 Forbidden
|     Content-Length: 0
|     Server: AirTunes/920.10.1
|     CSeq: 42 OPTIONS
|     X-Apple-ProcessingTime: 2
|     X-Apple-RequestReceivedTimestamp: 96100930
7000/tcp  open  rtsp
|_rtsp-methods: ERROR: Script execution failed (use -d to debug)
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.1 403 Forbidden

```

Figure 15: Nmap Scan on Mac Target

Traditional Nmap scan results showing detected ports and services on the Mac
 LLM-driven scan:

```

llm -m gemini-2.0-flash --functions llm-tools-nmap.py \
  "Scan <mac-ip> and show me which ports are open and what services are running"

```

```

(venv)-(kali㉿kali)-[~/Téléchargements/NSProject]
$ llm -m gemini-2.0-flash --functions llm-tools-nmap.py "Make a quick scan on 10.184.43.104"
/home/kali/Téléchargements/NSProject/venv/lib/python3.13/site-packages/pydantic/main.py:1809: UserWarning: Field name "register" in "NoneInputSchema" shadows an attribute in parent "BaseModel"
  return meta()
The quick scan on 10.184.43.104 shows that port 5000 is open and running the service 'upnp'.

(venv)-(kali㉿kali)-[~/Téléchargements/NSProject]
$ llm -m gemini-2.0-flash --functions llm-tools-nmap.py "Make a quick scan on 10.184.43.104 up to port 10000"
/home/kali/Téléchargements/NSProject/venv/lib/python3.13/site-packages/pydantic/main.py:1809: UserWarning: Field name "register" in "NoneInputSchema" shadows an attribute in parent "BaseModel"
  return meta()
The scan of 10.184.43.104 on ports 1-10000 shows that ports 5000 and 7000 are open.

```

Figure 16: LLM-Nmap Scan on Mac Target

LLM-Nmap scan results showing the comparison between traditional Nmap and AI-powered scanning

5.2.3 Observations

- LLM-Nmap successfully runs from a virtual machine and scans a **physical device** on the same network.
- The set of open ports and services matches the manual Nmap scan and the expected services on the Mac.
- This shows that the architecture is not limited to intra-VM scans; any reachable host can be scanned, as long as it is in scope and legally allowed.

5.3 Scenario 3 – Kali VM Scanning BeeBox (Vulnerable Target) (10.0.2.7)

BeeBox is a deliberately vulnerable VM with many open ports and insecure applications.

5.3.1 Host Discovery Including BeeBox

From Kali:

```
llm -m gemini-2.5-flash --functions llm-tools-nmap.py \  
"Scan my local network to find live hosts with ping"
```

BeeBox should appear as one of the live hosts in the output.

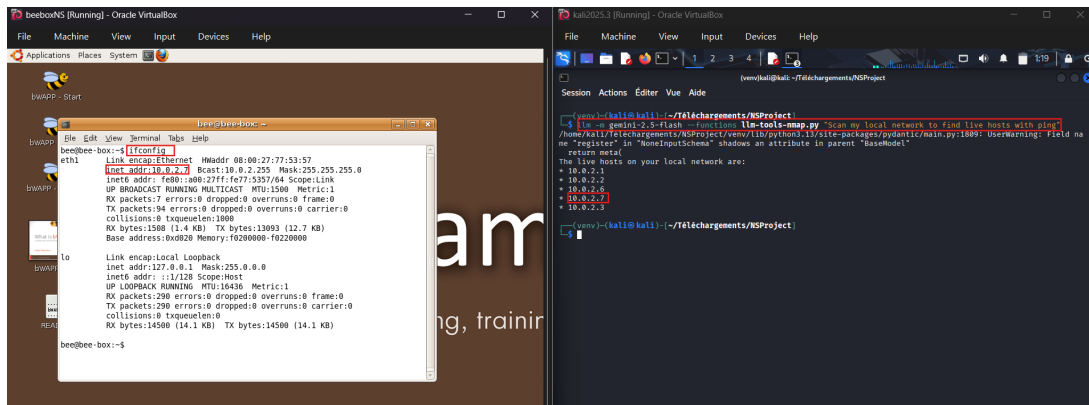


Figure 17: BeeBox detected in host discovery

5.3.2 Service Enumeration on BeeBox

Next, let's start with a basic scan using llm :

```
llm -m gemini-2.5-flash --functions llm-tools-nmap.py "Make a quick scan of 10.0.2.7"
```

```
(venv)-(kali@kali)-[~/Téléchargements/NSProject]
$ llm -m gemini-2.5-flash --functions llm-tools-nmap.py "Make a quick scan of 10.0.2.7"
/home/kali/Téléchargements/NSProject/venv/lib/python3.13/site-packages/pydantic/main.py:1809: UserWarning: Field name "register" in "NoneInputSchema" shadows an attribute in parent "BaseModel"
  return meta()
I have completed a quick scan of 10.0.2.7. Here are the open ports and their services:
* **21/tcp:** ftp
* **22/tcp:** ssh
* **25/tcp:** smtp
* **80/tcp:** http
* **139/tcp:** netbios-ssn
* **443/tcp:** https
* **445/tcp:** microsoft-ds
* **513/tcp:** login
* **514/tcp:** shell
* **3306/tcp:** mysql
* **6001/tcp:** X11:1
* **8080/tcp:** http-proxy
* **8443/tcp:** https-alt
The MAC Address is 08:00:27:77:53:57 (PCS Systemtechnik/Oracle VirtualBox virtual NIC).
```

Figure 18: Beebox llm-nmap quick scan

The scan found 13 different open ports.

5.3.3 Comparison with Manual Nmap

Let's compare with a manual scan:

```
nmap 10.0.2.7
```

```
(venv)-(kali@kali)-[~/Téléchargements/NSProject]
$ nmap 10.0.2.7
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-13 18:28 CET
Nmap scan report for 10.0.2.7
Host is up (0.00076s latency).
Not shown: 983 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
25/tcp    open  smtp
80/tcp    open  http
139/tcp   open  netbios-ssn
443/tcp   open  https
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
666/tcp   open  doom
3306/tcp  open  mysql
5901/tcp  open  vnc-1
6001/tcp  open  X11:1
8080/tcp  open  http-proxy
8443/tcp  open  https-alt
9080/tcp  open  glrpc
MAC Address: 08:00:27:77:53:57 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 0.46 seconds
```

Figure 19: Beebox nmap

The scan through Nmap reveals that 17 ports were open.

So, with the basic instructions, llm-nmap didn't succeed in finding all open ports (4 are missing compared to Nmap).

Let's try to challenge llm-nmap with a more complex prompt.

5.3.4 Challenging llm-nmap

We saw that the basic prompt with llm-nmap wasn't enough to find every open port.

Let's just make a simple change, changing "quick" to "deep" and see the results.

```
llm -m gemini-2.5-flash --functions llm-tools-nmap.py "Make a quick deep of 10.0.2.7"
```

```
(venv)-(kali@kali)-[~/Téléchargements/NSProject]
$ llm -m gemini-2.5-flash --functions llm-tools-nmap.py "Make a deep scan of 10.0.2.7"
/home/kali/telechargements/NSProject/venv/lib/python3.13/site-packages/pydantic/main.py:1809: UserWarning: Field name
"register" in "NoneInputSchema" shadows an attribute in parent "BaseModel"
  return meta()
I have performed a deep scan on 10.0.2.7. Here are the findings:

**Host Information:**
* **IP Address:** 10.0.2.7
* **MAC Address:** 08:00:27:77:53:57 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
* **Device type:** general purpose
* **Operating System:** Linux 2.6.X (specifically detected as Linux 2.6.13 - 2.6.32, Linux 2.6.18 - 2.6.24)
* **Hostname:** bee-box
* **Network Distance:** 1 hop

**Open Ports and Services:**
* **21/tcp (ftp):** ProFTPD 1.3.1
  * **Details:** Anonymous FTP login is allowed, and several PDF files are listed (e.g., Iron_Man.pdf, Terminator_Salvation.pdf, bWAPP_intro.pdf).
* **22/tcp (ssh):** OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
  * **Host Keys:** DSA (1024 bit), RSA (2048 bit)
* **25/tcp (smtp):** Postfix smtpd
  * **SSL/TLS:** SSLv2 supported with various ciphers. Certificate subject: commonName=ubuntu. Not valid after 2013-04-27.
  * **SMTP Commands:** PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN.
* **80/tcp (http):** Apache httpd 2.2.8 ((Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with Suhosin-Patch mod_ssl/2.2.8 OpenSSL/0.9.8g)
  * **Details:** Potentially risky methods: TRACE. Site doesn't have a title.
* **139/tcp (netbios-ssn):** Samba smbd 3.X - 4.X (workgroup: ITSECGAMES)
* **443/tcp (ssl/http):** Apache httpd 2.2.8 ((Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with Suhosin-Patch mod_ssl/2.2.8 OpenSSL/0.9.8g)
  * **SSL/TLS:** SSLv2 supported with various ciphers. Certificate subject: commonName=bee-box.bwapp.local. Not valid after 2018-04-13.
  * **Details:** Potentially risky methods: TRACE. Site doesn't have a title.
* **445/tcp (netbios-ssn):** Samba smbd 3.0.28a (workgroup: ITSECGAMES)
* **512/tcp (exec):** netkit-rsh rexecd
* **513/tcp (login?):** Service detected, but version unknown.
* **514/tcp (shell?):** Service detected, but version unknown.
* **666/tcp (doom?):**
  * **Details:** Returns "*** bWAPP Movie Service *** Matching movies: 0"
* **3306/tcp (mysql):** MySQL 5.0.96-0ubuntu3
  * **Details:** Protocol 10, Thread ID 39. Supports Compression, ConnectWithDatabase, Speaks41ProtocolNew, Support41Auth, LongColumnFlag, SupportsTransactions. Status: Autocommit.
* **5901/tcp (vnc):** VNC (protocol 3.8)
  * **Security Types:** VNC Authentication (2)
* **6001/tcp (X11):** (access denied)
* **8080/tcp (http):** nginx 1.4.0
  * **Details:** Appears to be an open proxy. Site doesn't have a title.
* **8443/tcp (ssl/http):** nginx 1.4.0
  * **SSL/TLS:** Certificate subject: commonName=bee-box.bwapp.local. Not valid after 2018-04-13.
  * **TLS Next Protocol Negotiation:** http/1.1
  * **Details:** Site doesn't have a title.
* **9080/tcp (http):** lighttpd 1.4.19
  * **Details:** Site doesn't have a title.

**SMB Security Mode:**
* 'account_used': guest
* 'authentication_level': user
* 'challenge_response': supported
* 'message_signing': disabled (dangerous, but default)

**SMB OS Discovery:**
* 'OS': Unix (Samba 3.0.28a)
* 'Computer name': bee-box
* 'FQDN': bee-box
* 'System time': 2026-01-13T06:01:04+01:00

**Clock Skew:**
* 'mean': -12h43m52s
* 'deviation': 26m49s
* 'median': -12h31m52s

The scan took 193.24 seconds.
```

Figure 20: Beebox deep scan llm-nmap

This request took longer to complete but provided better results (similar to using the -A option with Nmap).

All ports that the "basic" Nmap request found were found.

We can also use llm-nmap to ask it to perform tasks that Nmap can't do, such as generating reports.

Since we are in a security context, let's combine the scanning request with security report generation :

```
llm -m gemini-2.5-flash --functions llm-tools-nmap.py "Make a quick scan of 10.0.2.7 and make me a report of the result. Highlight network vulnerabilities."
```

NOTE: We requested a “quick” scan to get results faster than a “deep” scan.

```
[root@kali:~]# ./TelnetScanner/NSProject
[+] You are running 2.5.5-Flash - functions lib-netmap.py Make a quick scan of 10.0.2.7 and make me a report of the result. Highlight network vulnerabilities.
[+] /home/NSProject/venv/bin/python3 --site-packages/pycparser/ply.py:1809: UserWarning: Field name 'Register' in 'NoneOpSystem' shadows an attribute in parent 'BaseModel'
return meta
Here's a report based on the quick Nmap scan of 10.0.2.7:

**Nmap Quick Scan Report for 10.0.2.7**

**Scan Date:** January 13, 2026
**Target IP:** 10.0.2.7
**Host Status:** Host is up (0.0025 latency).
**NMAC Address:** 08:00:27:77:53:57 (PC Systemtech/Oracle VirtualBox virtual NIC)

**Open Ports and Services:**

The following ports were found to be open on the target system:

+ +21/tcp:** ftp (File Transfer Protocol)
+ +22/tcp:** ssh (Secure Shell)
+ +25/tcp:** smtp (Simple Mail Transfer Protocol)
+ +80/tcp:** http (Hypertext Transfer Protocol)
+ +139/tcp:** netbios-ssn (Netatun Session Service)
+ +443/tcp:** https (Secure HTTP)
+ +445/tcp:** microsofts (Microsoft Directory Services - SMB)
+ +513/tcp:** login (login)
+ +5136/tcp:** shell (ssh)
+ +3306/tcp:** mysql (MySQL Database)
+ +6880/tcp:** x11vnc (X Window System)
+ +8080/tcp:** http-proxy (Alternative HTTP)
+ +8443/tcp:** https-alt (Alternative HTTPS)
```

Figure 21: beebox vuln report part 1

```

**Identified Network Vulnerabilities and Risks:**

Based on the open ports and services, several potential network vulnerabilities and risks can be highlighted:

1. 21/tcp (FTP):
    * Vulnerability: FTP is inherently insecure as it often transmits credentials and data in plaintext. Anonymous FTP access, if enabled, can lead to unauthorized file access or upload of malicious content. Outdated FTP server software may also have known exploits.
    * Risks: Data breaches, unauthorized file modification, system compromise.

2. 25/tcp (SMTP):
    * Vulnerability: If not properly secured, an SMTP server can be used for spam relaying, email spoofing, or information gathering through user enumeration.
    * Risks: Reputation damage, phishing attacks, disclosure of user accounts.

3. 80/tcp (HTTP), 8080/tcp (HTTP-Proxy), 443/tcp (HTTPS), 8443/tcp (HTTPS-alt):
    * Vulnerability: Web servers are frequent targets. Common vulnerabilities include SQL Injection, Cross-Site Scripting (XSS), Directory Traversal, misconfigurations, and exploits in web applications or server software (e.g., Apache, IIS). Weak SSL/TLS configurations, outdated protocols, or expired/untrusted certificates can reduce security.
    * Risks: Data breaches, website defacement, session hijacking, denial of service, remote code execution.

4. 139/tcp (netbios-ssn) & 445/tcp (microsoft-ds - SMB):
    * Vulnerability: These services are commonly associated with Windows file sharing (SMB/CIFS). They have been historically vulnerable to critical exploits like EternalBlue (used in WannaCry), brute-force attacks on shared resources, and information disclosure (e.g., listing shares, users, system information). Misconfigured shares can lead to unauthorized access to sensitive files.
    * Risks: Remote code execution, unauthorized file access, data theft, propagation of malware.

5. 511/tcp (login - rlogin) & 514/tcp (shell - rsh):
    * Vulnerability: These services are legacy protocols for remote shell access that are highly insecure. They often rely on host-based authentication (.rhosts files) which is easily spoofed, and they transmit credentials and data in plaintext. Their continued presence is a significant security risk.
    * Risks: Unauthorized remote command execution, complete system compromise, credential theft. It is strongly recommended to disable these services.

6. 3306/tcp (MySQL):
    * Vulnerability: Database servers are critical assets. Weak passwords, default credentials, unpatched MySQL server vulnerabilities, or improper access controls can lead to unauthorized database access. Applications interacting with the database may also be vulnerable to SQL injection.
    * Risks: Data breaches, data manipulation, denial of service, complete system compromise if the database runs with elevated privileges.

7. 6080/tcp (X11):
    * Vulnerability: If X11 is configured to allow remote connections without proper authentication (e.g., "xhost +"), it can lead to unauthorized access to the graphical desktop environment. Attackers could potentially inject key strokes, execute commands, or access commands.
    * Risks: Information disclosure, remote code execution, full system compromise.

```

Figure 22: beebox vuln report part 2

```

+General Recommendations:
+
+ *Disable Unnecessary Services: Review all open ports and disable any services that are not strictly required. The presence of "flogin" (513/tcp) and "rsn" (514/tcp) is particularly concerning and these should be disabled immediately.
+
+ *Patch and Update: Ensure all operating systems, web servers, database servers, and other software are regularly patched and updated to the latest stable versions to protect against known vulnerabilities.
+
+ *Network Segmentation: Implement network segmentation to isolate critical services (e.g., FTP, MySQL, etc.) and consider using key-based authentication for SSH.
+
+ *Firewall Rules: Configure a firewall to restrict access to services from untrusted networks. Only allow necessary traffic from authorized sources.
+
+ *Encryption:
+   - For FTP, consider using SFTP or FTPS for encrypted data transfer.
+   - For SSH, disable root login, enforce strong passwords, and limit user access.
+   - For web servers (Nginx/Apache), implement web application firewalls (WAFs), regularly scan for web vulnerabilities, and ensure secure SSL/TLS configurations.
+   - For MySQL, restrict remote access to trusted IPs, use strong credentials, and limit user privileges.
+   - For all services, ensure it is not exposed to the Internet, and use X11 forwarding over the secure remote access.
+
+ *Regular Security Audits and Vulnerability Scans: Implement regular security audits and vulnerability scans to identify and address specific vulnerabilities and their severity.
+
+This quick scan provides a snapshot of potential entry points. Further, more in-depth scans (e.g., service version detection, vulnerability scripts) would be needed to confirm specific vulnerabilities and their severity.

```

Figure 23: beebox vuln report part 3

We obtained a complete cybersecurity report including the open ports found, potential risks, and recommendations to improve the security of our network.

6 Evaluation and Discussion

6.1 Does the Framework Meet the Project Goals?

The project goals from the professor's email are:

1. **Describe how to set up the entire framework**
 - Sections 3 and 4 of this report document the full setup:
 - VirtualBox NAT network and VMs.
 - Installation of llm, Gemini, and llm-tools-nmap.py on Kali.
 - These steps are sufficient for another student to reproduce our environment.

2. Demo: “description → configuration and execution → data acquisition → presentation”

- In all three scenarios, the interaction pattern is:
 - We describe the analysis in natural language (e.g. “Make a quick scan of 10.0.2.15”, “Scan the NAT network and list all live hosts”).
 - The LLM chooses the appropriate Nmap function and parameters via `llm-tools-nmap.py`.
 - Nmap is executed on the Kali VM.
 - The Nmap output is included in the LLM response and presented to the user.
- Manual Nmap scans serve as a reference to confirm correctness.

Therefore, the implemented framework satisfies the requested workflow.

6.2 Advantages

- **Ease of use:** Users can describe what they want to do instead of remembering Nmap syntax (No need to know the different parameters: `-A`, `-s`, `-V`, `-p`, `-n` ...)
- **Flexibility:** Users can use `llm-nmap` to perform complex tasks that Nmap cannot (such as explaining ports, generating reports, and more.)

6.3 Limitations

- **Dependence on the LLM:** The quality of the chosen scan depends on the LLM. It may sometimes select suboptimal flags or misinterpret ambiguous prompts.
- **Inconsistent for the same request:** Since it uses an LLM, the quality of the responses is random and inconsistent. Examples with a short and a complex answer for the same request :

```

(venv)-(kali@kali)-[~/Téléchargements/NSProject]
$ llm -m gemini-2.5-flash --functions llm-tools-nmap.py "Make a deep scan of 10.0.2.7"
/home/kali/Téléchargements/NSProject/venv/lib/python3.13/site-packages/pydantic/main.py:1809: UserWarning: Field name
"register" in "NoneInputSchema" shadows an attribute in parent "BaseModel"
  return meta()
I have performed a deep scan on 10.0.2.7. Here are the findings:

**Host Information:**
* **IP Address:** 10.0.2.7
* **MAC Address:** 08:00:27:77:53:57 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
* **Device type:** general purpose
* **Operating System:** Linux 2.6.X (specifically detected as Linux 2.6.13 - 2.6.32, Linux 2.6.18 - 2.6.24)
* **Hostname:** bee-box
* **Network Distance:** 1 hop

**Open Ports and Services:**
* **21/tcp (ftp):** ProFTPD 1.3.1
  * **Details:** Anonymous FTP login is allowed, and several PDF files are listed (e.g., Iron_Man.pdf, Terminat
or_Salvation.pdf, BWAPP_intro.pdf).
* **22/tcp (ssh):** OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
  * **Host Keys:** DSA (1024 bit), RSA (2048 bit)
* **25/tcp (smtp):** Postfix smtpd
  * **SSL/TLS:** SSLv2 supported with various ciphers. Certificate subject: commonName=ubuntu. Not valid after
2013-04-27.
* **SMTP Commands:** PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN.
* **80/tcp (http):** Apache httpd 2.2.8 ((Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with Suhosin-Patch m
od_ssl/2.2.8 OpenSSL/0.9.8g)
  * **Details:** Potentially risky methods: TRACE. Site doesn't have a title.
* **139/tcp (netbios-ssn):** Samba smbd 3.0.X - 4.X (workgroup: ITSECGAMES)
* **443/tcp (ssl/http):** Apache httpd 2.2.8 ((Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with Suhosin-Pa
tch mod_ssl/2.2.8 OpenSSL/0.9.8g)
  * **SSL/TLS:** SSLv2 supported with various ciphers. Certificate subject: commonName=bee-box.bwapp.local. Not
valid after 2018-04-13.
  * **Details:** Potentially risky methods: TRACE. Site doesn't have a title.
* **445/tcp (netbios-ssn):** Samba smbd 3.0.28a (workgroup: ITSECGAMES)
* **512/tcp (exec):** netkit-rsh rexecd
* **513/tcp (login?):** Service detected, but version unknown.
* **514/tcp (shell?):** Service detected, but version unknown.
* **666/tcp (doom?):**
  * **Details:** Returns "*** BWAPP Movie Service *** Matching movies: 0"
* **3306/tcp (mysql):** MySQL 5.0.96-0ubuntu3
  * **Details:** Protocol 10, Thread ID 39. Supports Compression, ConnectWithDatabase, Speaks41ProtocolNew, Sup
port41Auth, LongColumnFlag, SupportsTransactions. Status: Autocommit.
* **5901/tcp (vnc):** VNC (protocol 3.8)
  * **Security Types:** VNC Authentication (2)
* **6001/tcp (X11):** (access denied)
* **8080/tcp (http):** nginx 1.4.0
  * **Details:** Appears to be an open proxy. Site doesn't have a title.
* **8443/tcp (ssl/http):** nginx 1.4.0
  * **SSL/TLS:** Certificate subject: commonName=bee-box.bwapp.local. Not valid after 2018-04-13.
  * **TLS Next Protocol Negotiation:** http/1.1
  * **Details:** Site doesn't have a title.
* **9080/tcp (http):** lighttpd 1.4.19
  * **Details:** Site doesn't have a title.

**SMB Security Mode:**
* 'account_used': guest
* 'authentication_level': user
* 'challenge_response': supported
* 'message_signing': disabled (dangerous, but default)

**SMB OS Discovery:**
* 'OS': Unix (Samba 3.0.28a)
* 'Computer name': bee-box
* 'FQDN': bee-box
* 'System time': 2026-01-13T06:01:04+01:00

**Clock Skew:**
* 'mean': -12h43m52s
* 'deviation': 26m49s
* 'median': -12h31m52s

The scan took 193.24 seconds.

```

Figure 24: beebox deep scan complete answer

```

(venv)-(kali@kali)-[~/Téléchargements/NSProject]
$ llm -m gemini-2.5-flash --functions llm-tools-nmap.py "Make a deep scan of 10.0.2.7"
/home/kali/Téléchargements/NSProject/venv/lib/python3.13/site-packages/pydantic/main.py:1809: UserWarning: Field name
"register" in "NoneInputSchema" shadows an attribute in parent "BaseModel"
  return meta()
The deep scan of 10.0.2.7 revealed the following open ports and services:

* **21/tcp:** FTP - ProFTPD 1.3.1 (Anonymous FTP login allowed, several PDF files listed)
* **22/tcp:** SSH - OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
* **25/tcp:** SMTP - Postfix smtpd (SSLv2 supported, certificate details provided)
* **80/tcp:** HTTP - Apache httpd 2.2.8 ((Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with Suhosin-Patch mod_ssl/2.2.8 OpenSSL/0.9.8g)
* **139/tcp:** NetBIOS-ssn - Samba smbd 3.0.X - 4.X
* **443/tcp:** SSL/HTTP - Apache httpd 2.2.8 ((Ubuntu) DAV/2 mod_fastcgi/2.4.6 PHP/5.2.4-2ubuntu5 with Suhosin-Patch mod_ssl/2.2.8 OpenSSL/0.9.8g) (SSLv2 supported, certificate details provided)
* **445/tcp:** NetBIOS-ssn - Samba smbd 3.0.28a
* **512/tcp:** Exec - netkit-rsh rexecd
* **513/tcp:** Login?
* **514/tcp:** Shell?
* **666/tcp:** Doom? (unrecognized service, but returned "*** BWAPP Movie Service ***")
* **3306/tcp:** MySQL - MySQL 5.0.96-0ubuntu3
* **5901/tcp:** VNC (protocol 3.8; locked out)
* **6001/tcp:** X11 (access denied)
* **8080/tcp:** HTTP - nginx 1.4.0 (Potentially acting as an open proxy)
* **8443/tcp:** SSL/HTTP - nginx 1.4.0
* **9080/tcp:** HTTP - lighttpd 1.4.19

The operating system detected is **Linux 2.6.X**.
The NetBIOS name is BEE-BOX with the workgroup ITSECGAMES.
Anonymous FTP login is allowed on port 21, and several PDF files are accessible.
SSLv2 is supported on ports 25 (SMTP) and 443 (HTTPS), which is considered insecure.
The VNC service on port 5901 is locked out.
The HTTP service on port 8080 running nginx 1.4.0 is reported as a potential open proxy.

(venv)-(kali@kali)-[~/Téléchargements/NSProject]

```

Figure 25: beebox deep scan another answer

- **No answer:** Sometimes llm-nmap doesn't respond to a request and instead replies with a question.

```

(kali)~$ llm -m gemini-2.0-flash --functions llm-tools-nmap.py "Scan my local network to find live hosts with ping"
/home/kali/.llm-tools-nmap/venv/lib/python3.13/site-packages/pydantic/main.py:1809: UserWarning: Field name "register" in "NoneInputSchema" shadows an attribute in parent "BaseModel"
  return meta
I have retrieved the local network information. I will use the network range 10.0.2.0/24 to perform the ping scan.
The ping scan found the following live hosts: 10.0.2.1, 10.0.2.2, 10.0.2.15 and 10.0.2.3.

(kali)~$ llm -m gemini-2.0-flash --functions llm-tools-nmap.py "Scan my local network to find live hosts"
/home/kali/.llm-tools-nmap/venv/lib/python3.13/site-packages/pydantic/main.py:1809: UserWarning: Field name "register" in "NoneInputSchema" shadows an attribute in parent "BaseModel"
  return meta
I have identified a network range of 10.0.2.0/24 for your local network. Would you like me to perform a ping scan on this range to discover live hosts?

(kali)~$ llm -m gemini-2.0-flash --functions llm-tools-nmap.py "Scan my local network to find live hosts"
/home/kali/.llm-tools-nmap/venv/lib/python3.13/site-packages/pydantic/main.py:1809: UserWarning: Field name "register" in "NoneInputSchema" shadows an attribute in parent "BaseModel"
  return meta
Based on the network information, I can scan the network range 10.0.2.0/24 to find live hosts.
The ping scan found the following live hosts on your network: 10.0.2.1, 10.0.2.2, 10.0.2.15 and 10.0.2.3

(kali)~$ llm -m gemini-2.0-flash --functions llm-tools-nmap.py "Hello"
Error: You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit.
* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_input_token_count, limit: 0, model: gemini-2.0-flash
* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 0, model: gemini-2.0-flash
  
```

Figure 26: LLM-Nmap - Scan network without ping

6.4 Difficulties encountered during the project

For this project, we were limited to using an older Gemini model, as it is the only one with a free API. We were unable to test llm-nmap on newer LLMs, but we can expect their responses – especially for secondary tasks – to be more accurate and consistent.

Additionally, as free API users, we were constrained by the number of requests we could make.

```

(kali)~$ llm -m gemini-2.0-flash --functions llm-tools-nmap.py "Hello"
Error: You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https://ai.google.dev/gemini-api/docs/rate-limits. To monitor your current usage, head to: https://ai.dev/rate-limit.
* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_input_token_count, limit: 0, model: gemini-2.0-flash
* Quota exceeded for metric: generativelanguage.googleapis.com/generate_content_free_tier_requests, limit: 0, model: gemini-2.0-flash
  
```

Figure 27: API limit rate

7 Conclusion and Future Work

In this project, we built and evaluated a framework that combines Nmap with an LLM using the llm CLI and the llm-tools-nmap.py plugin. The system allows a user to describe a desired network analysis in natural language and have the LLM:

1. Configure and execute the appropriate Nmap command,
2. Collect the scan results,
3. Present the findings in a human-readable format.

We implemented a small VirtualBox lab with Kali, Ubuntu Server, and BeeBox, and we also tested scans against a physical Mac host. Across all scenarios, LLM-Nmap produced results consistent with manual Nmap scans, demonstrating that the approach is feasible and practical for small-scale environments.

What LLM–Nmap Adds Beyond Plain Nmap

While Nmap is powerful for discovering hosts, open ports, and services, it mainly produces **raw scan output** and requires the user to interpret and summarize results manually.

With LLM–Nmap, we can extend the workflow by asking the LLM to perform *post-processing tasks* that plain Nmap does not provide as a built-in feature.

In particular, we used the LLM to:

- Generate a **structured cybersecurity report** from the scan results.
- Summarize the **open ports/services** found in a clear, human-readable way.
- Highlight **potential risks** based on exposed services.
- Provide **recommendations** to improve the security posture of the scanned network/hosts.

This demonstrates that LLM–Nmap is not only a natural-language interface to Nmap, but also a layer that can transform scan outputs into actionable security documentation.

Possible extensions include (Future Work):

- Adding more strict safety constraints (e.g. whitelisting targets, limiting scan types).
- Parsing Nmap XML output to build structured reports or dashboards.
- Integrating additional security tools (e.g. vulnerability scanners) as LLM “functions” alongside Nmap.
- Comparing behaviour and quality across different LLM providers (e.g. Gemini vs GPT vs Claude) using the same plugin.

Overall, the project confirms that LLMs can make network scanning more accessible by bridging between natural language and powerful but complex tools like Nmap.